From Pixels to Decisions: Five Methods for Robot Vision

Harry He
ENGR 6010 AI in Robotics, First Test, Question 1 Instructor: Dr. Hongbo Zhang

Abstract. A robot with a camera has to turn raw pixel values into decisions. This paper reviews five methods that each do part of this job: color spaces, Gaussian pyramids, optical flow, GrabCut and supervised learning. For each one I explain the main idea and the assumption behind it, using the papers I read in this course. I also add results from my own assignments. Each method fails in a clear way when its assumption breaks.

Keywords: color space, Gaussian pyramid, optical flow, GrabCut, supervised learning, lasso

1 Introduction

A legged robot such as the Unitree Go2 can walk through a building, but it only knows what its sensors tell it. Its camera gives a grid of numbers, three for each pixel. Before the robot can follow a person or avoid an obstacle, these numbers must be organized and understood.

The five methods in this review work as a chain. A color space decides what the three numbers of a pixel mean. A Gaussian pyramid stores the image at several sizes. Optical flow compares two frames to measure motion. GrabCut separates an object from its background. Supervised learning trains a model on labeled examples so that it can decide about new data. For each method I describe the theory, cite the sources, and give its main limit. Where I tested a claim myself, I report the number I measured. Section 2 reviews the methods, Section 3 compares them, and Section 4 concludes.

2 Review of the Methods

2.1 Color Space

A color space, or color model, is the way a color is stored as numbers. Ibraheem et al. [1] divide color into luminance, which is brightness, and chrominance, which is hue and saturation. RGB stores red, green and blue directly. Cameras and screens use it because no conversion is needed, but brightness is mixed into all three channels. HSI and HSV keep brightness in one channel and color in the other two, so hue alone can be used for segmentation. YCbCr keeps brightness in Y and two color differences in Cb and Cr, and it is used for digital video. CIE L*a*b* is device-independent, so the same numbers mean the same color on any device.

No color space is best for every task. The review warns that hue is an angle, so it becomes unstable near gray. In Reading 2 I moved every pixel by one RGB unit. The least saturated pixels changed hue 2.6 to 123 times more than the most saturated ones, depending on the photo. I also tested whether YCbCr and Lab make GrabCut more stable under new lighting. The ranking changed when I summarized the same runs in another fair way, so I could not support that claim.

2.2 Gaussian Pyramid

A Gaussian pyramid stores one image at several resolutions. Burt and Adelson [2] build it by repeating one step. Each value in the next level is a weighted average of a 5 by 5 window in the level below. Then only every second row and column is kept. Each level therefore has half the width and half the height of the one before. In their example, a 257 by 257 image is only 9 by 9 at level 5. The weights give the pyramid its name. When the center weight is 0.4, the repeated averaging is very close to a Gaussian blur. The paper then subtracts neighboring levels to build a Laplacian pyramid for image compression.

For robots, the main use is motion. Differential optical flow only works for small motion [3]. In Homework 1, my own Lucas-Kanade code had an error of 0.0001 pixels for a 1-pixel shift, but 1.77 pixels for a 5-pixel shift. Bouguet [4] fixes this by tracking at the smallest level first, then doubling the result and refining it on each larger level. For a pyramid of height L, the largest trackable motion grows by a factor of $2^{L+1} - 1$. For $L = 3$ this factor is 15. The cost is that a small object moving fast can disappear in the coarse levels [3].

2.3 Optical Flow

Horn and Schunck [5] define optical flow as the apparent velocity of brightness patterns in an image. It can differ from the true motion, because shading and reflections do not always move with the object. Most methods assume brightness constancy, which means a point keeps its brightness while it moves. A first-order Taylor expansion then gives

$$I_x u + I_y v + I_t = 0, \quad (1)$$

where I_x and I_y are the image gradients, I_t is the change over time and (u, v) is the flow. This is one equation with two unknowns, so only the motion along the gradient can be found. This is called the aperture problem.

Each method adds a second assumption. Horn and Schunck assume the flow is smooth over the whole image. Lucas and Kanade [6] assume that all pixels in a small window share one displacement. One equation per pixel, solved by least squares, gives a 2 by 2 system $A^TA\ v = A^Tb$. Shi and Tomasi [7] accept a window only when the smaller eigenvalue of A^TA is above a threshold. Corners pass this test, while straight edges and flat areas fail. In Homework 1 the 3 by 3 patch was a pure ramp, so $\det(A^TA) = 0$ and only the normal flow could be found. In Reading 1 my webcam tracker put its corners on my eyes and shirt seams, and almost none on the plain wall. Fortun et al. [3] add the other common failures: lighting change, occlusion and large motion.

2.4 GrabCut

GrabCut [8] cuts an object out of its background with very little user input. The user only draws a rectangle. Pixels outside it are fixed as background, and pixels inside start as probable foreground. The method minimizes an energy E = U + V. The data term U measures how well each pixel's color fits the color model of its label. Each model is a Gaussian mixture in RGB with five components. The smoothness term V adds a penalty when two neighboring pixels get different labels. This penalty is smaller where their colors already differ, so the cut prefers real edges. For fixed color models, a graph cut finds the exact minimum of E. GrabCut then refits the models from the new labels and cuts again, and E goes down at every step.

The paper reports an error of 2.13%, against 1.36% for a graph cut where the user marks both regions. The gain is less user work, not higher accuracy. Section 5 lists three failure cases: low boundary contrast, camouflage, and background inside the rectangle that is missing outside it. In Reading 2 I tested camouflage on an image with a known mask. The cut stayed above IoU 0.96, then fell to 0 in one step when the contrast-to-noise ratio dropped to about 2.2. I also found that OpenCV's GrabCut starts from a random state. Without a fixed seed, repeated runs on one photo matched the first run at an IoU as low as 0.651.

2.5 Supervised Learning

Supervised learning trains a model on examples with known labels, and then labels new data. Kotsiantis [9] describes the steps as collecting data, cleaning it, choosing features, training a classifier and testing it. Testing uses a held-out set or cross-validation. The review compares decision trees, neural networks, naive Bayes, k-nearest neighbors and support vector machines. It concludes that no single algorithm is best on every dataset. It also explains bias and variance. A simple model can miss the pattern, and a flexible model can overfit the training data.

The lasso [10] limits overfitting in linear regression. It minimizes the squared error while the sum of the absolute coefficients stays below a limit t. With orthonormal inputs, each coefficient moves toward zero by a fixed amount and stops at zero. Ridge regression multiplies each coefficient by a factor instead, so it never reaches zero. In two dimensions the lasso limit is a diamond with corners on the axes. The solution often lands on a corner, where one coefficient is exactly zero. So the lasso shrinks and selects features in one step. It is not always the best choice. Section 11 of the paper shows that ridge wins when there are many small effects. scikit-learn [11] writes the same problem with a penalty alpha instead of the limit t.

In Reading 3 I used the lasso on 25 color and texture features of each pixel to separate a weed from soil. The strongest penalty kept only blue, saturation and Lab-a, with a pixel F1 of 0.781 against 0.786 for all 25. But cleaning the training images with these masks did not help a YOLO11n detector. It scored 0.952 mAP@0.5 against 0.960, while the plain model alone moved by 0.007 between repeated runs.

3 Comparison

Table 1 puts the five methods side by side. The last column is what I measured myself in Homework 1 and Readings 1 to 3.

Table 1. The five methods, the assumption each one needs, and where it breaks.

Method	Main assumption	Breaks when	What I measured
Color space	The chosen channels separate what matters	Hue is used on grayish pixels	Hue moved up to 123 times more on the least saturated pixels
Gaussian pyramid	Coarse levels keep the large structures	A small object moves fast	Without a pyramid, Lucas-Kanade error was 1.77 px for a 5 px shift
Optical flow	Brightness stays constant and motion is small	Edges, flat areas, lighting change, occlusion	$\det(A^TA) = 0$ on a ramp patch
GrabCut	Object and background have different colors	Camouflage or low boundary contrast	Failure below a contrast-to-noise ratio of about 2.2
Supervised learning	Test data looks like the training data	Training images differ from real ones	Training on box-masked images dropped mAP@0.5 to 0.45

4 Conclusion

The five methods form a chain from raw pixels to a decision, and they depend on each other. GrabCut builds its color models in RGB. In robot vision the pyramid is mainly used to extend optical flow. My lasso features came from four color spaces. Each method also rests on one simple assumption, as Table 1 shows.

My own tests taught me two things that the papers do not say directly. First, failure can be sudden. GrabCut worked well until the contrast-to-noise ratio reached about 2.2, and then it failed completely. Second, a result must be compared with the run-to-run noise of the method. Without that check, the drop from 0.960 to 0.952 in Reading 3 would have looked like a real finding. The next step is to run these methods on camera data from the Unitree Go2. There, camera motion, changing light and cluttered floors will test all five assumptions at once.

References

[1] N. A. Ibraheem, M. M. Hasan, R. Z. Khan, and P. K. Mishra, “Understanding color models: A review,” ARPN Journal of Science and Technology, vol. 2, no. 3, pp. 265–275, 2012.

[2] P. J. Burt and E. H. Adelson, “The Laplacian pyramid as a compact image code,” IEEE Transactions on Communications, vol. 31, no. 4, pp. 532–540, 1983.

[3] D. Fortun, P. Bouthemy, and C. Kervrann, “Optical flow modeling and computation: A survey,” Computer Vision and Image Understanding, vol. 134, pp. 1–21, 2015.

[4] J.-Y. Bouguet, “Pyramidal implementation of the Lucas Kanade feature tracker: Description of the algorithm,” Intel Corporation, Microprocessor Research Labs, 2000.

[5] B. K. P. Horn and B. G. Schunck, “Determining optical flow,” Artificial Intelligence, vol. 17, pp. 185–203, 1981.

[6] B. D. Lucas and T. Kanade, “An iterative image registration technique with an application to stereo vision,” in Proc. 7th International Joint Conference on Artificial Intelligence (IJCAI), Vancouver, 1981, pp. 674–679.

[7] J. Shi and C. Tomasi, “Good features to track,” in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 1994, pp. 593–600.

[8] C. Rother, V. Kolmogorov, and A. Blake, “GrabCut: Interactive foreground extraction using iterated graph cuts,” ACM Transactions on Graphics, vol. 23, no. 3, pp. 309–314, 2004.

[9] S. B. Kotsiantis, “Supervised machine learning: A review of classification techniques,” Informatica, vol. 31, pp. 249–268, 2007.

[10] R. Tibshirani, “Regression shrinkage and selection via the lasso,” Journal of the Royal Statistical Society, Series B, vol. 58, no. 1, pp. 267–288, 1996.

[11] scikit-learn developers, “1. Supervised learning,” scikit-learn 1.9.1 User Guide. https://scikit-learn.org/stable/supervised_learning.html

Technical Note: Getting Ready to Work with the Unitree Go2

Harry He

ENGR 6010 AI in Robotics, First Test, Questions 2 to 4 Instructor: Dr. Hongbo Zhang

Summary

I set up an Ubuntu 22.04 virtual machine on my Mac, installed the Unitree SDK, and ran three SDK examples against a simulated Go2. I ran Go2Py, one of the five demo projects, with its trained walking controller. I built a small test field in the simulator, recorded 170 camera frames with exact labels and depth, and ran the five methods from Question 1 on them. Finally I ran a Go2 walking policy from unitree_rl_gym in MuJoCo.

Along the way I found four problems and fixed them. Go2Py's front camera looks out of the dog's side. The simulator's default time step makes the standing dog slide away. A sim-to-sim script makes the dog fall when it turns. OpenCV's Farneback flow returns almost zero on weak texture. Each one is described below with the measurement that showed it. Table 1 shows what is finished and what still needs the real robot in the lab.

Table 1. What I did for each question.

Question	What ran	Main result	Still needs the lab
Q2 Linux and SDK	Ubuntu 22.04 VM in UTM, CycloneDDS 0.10.5, unitree_sdk2py 1.0.1, three examples on unitree_mujoco	the simulated dog stands up (body 0.117 m to 0.384 m)	sport-mode examples on the real dog
Q3 Dog apps	Go2Py MuJoCo simulator with its walk-these-ways RL controller	follows forward, turn, sideways and backward commands within 0.01 m/s	the four ROS 2 / WebRTC projects
Q4 Data and simulation	170 synthetic camera frames, the five methods, unitree_rl_gym Go2 policy in MuJoCo	Sections 3.2 to 3.4	real camera data (script ready)

1 Question 2: Getting the Dog Environment Ready

1.1 Linux platform

The course guide recommends Ubuntu 20.04 or 22.04, and a virtual machine for a Mac. My Mac has an Apple M4 Pro chip, so I used UTM with the ARM64 version of Ubuntu 22.04. Instead of clicking through the installer, I used the Ubuntu cloud image and a small cloud-init file. On first boot it creates my user, adds my SSH key and installs the build tools. The VM has 4 cores, 8 GB of memory and a 32 GB disk. The whole setup is one script (q2_sdk/setup_vm.sh), so it can be repeated.

It took three tries to boot correctly. Both problems were in how the configuration disk reached Ubuntu.

1. UTM created the virtual CD drive but did not insert my configuration image, so Ubuntu started with default settings and my user did not exist.
2. My image, made with hdiutil -hfs, started with an Apple partition map. Linux read it as a partitioned disk and never saw the “cidata” label. A plain ISO 9660 image attached as a second disk worked.

1.2 Unitree SDK

unitree_sdk2_python talks to the robot through CycloneDDS 0.10. There is no ready-made CycloneDDS package for Linux on ARM, so I built the C library from source (0.10.5) and pointed the Python package to it with CYCLONEDDS_HOME. Then pip3 install -e . failed with “can't create or remove files in install directory”. Ubuntu 22.04 ships setuptools 59, which falls back to an old install method that writes into /usr/local. Upgrading pip and setuptools first and installing with --user fixed it.

1.3 Three examples

Without the robot I used unitree_mujoco, Unitree's own simulator. It sends and receives the same DDS messages as the real Go2, so the SDK examples run against it with small changes.

1. helloworld publisher and subscriber. 12 messages were sent and 12 were received with the same time stamps, so DDS works inside the VM.
2. go2_stand_example.py. I changed three things and left the control code alone. It uses the simulator's DDS domain 1 on the loopback interface. SportClient and MotionSwitcher are removed, because these services run on the robot and do not exist in the simulator. The “press Enter” line is removed too. The dog stood up from 0.117 m to 0.384 m and lowered again, as the example intends (Figure 1).
3. wireless_controller.py. I changed it to the Go2 message type and the simulator's topic. It printed 1,933 remote-controller states read from rt/lowstate.

A separate recorder saved 9,932 rt/lowstate messages at about 495 Hz during the run. During the stand the body moved less than 2 cm sideways or forward.

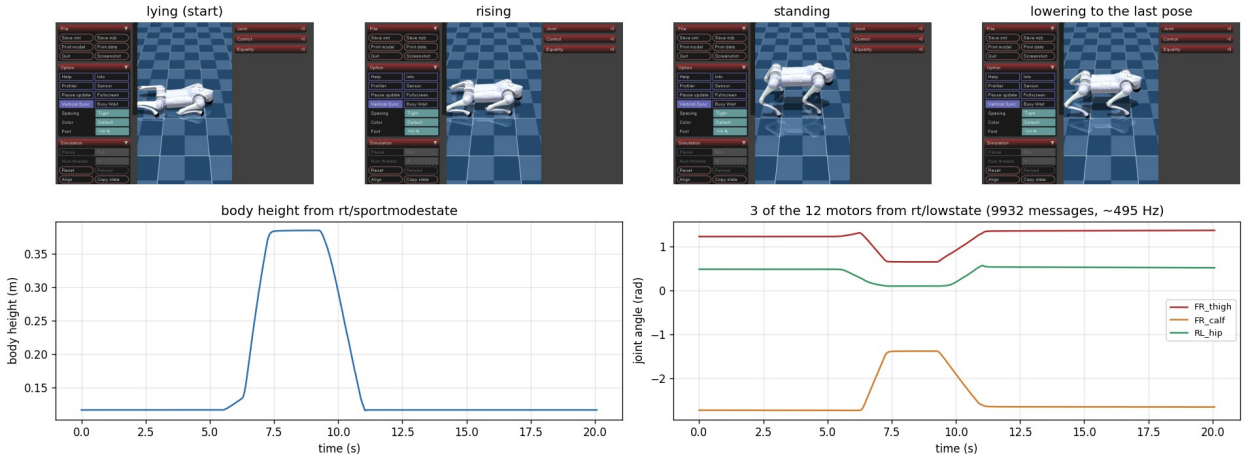


Figure 1. *The stand example in the simulator. Top: screenshots from the VM. Bottom: body height and three of the twelve motor angles, read through the SDK while the example ran.*

1.4 A problem I found: the standing dog slid away

The first time, the dog stood up and then disappeared from the screen. The recorded position showed it sliding backwards at about 1 m/s while every joint target was fixed. I repeated the example's joint targets directly in MuJoCo, with no DDS and no threads, and changed only the physics time step (Figure 2). With the simulator's default step of 5 ms the body moved 3.01 m in 6 s. With 2 ms or 1 ms it moved 0.01 m. The stand example uses stiff gains (kp = 60, kd = 5), and a 5 ms contact step is too coarse for them. The real robot's control period is 2 ms, so I set the simulator to 2 ms.

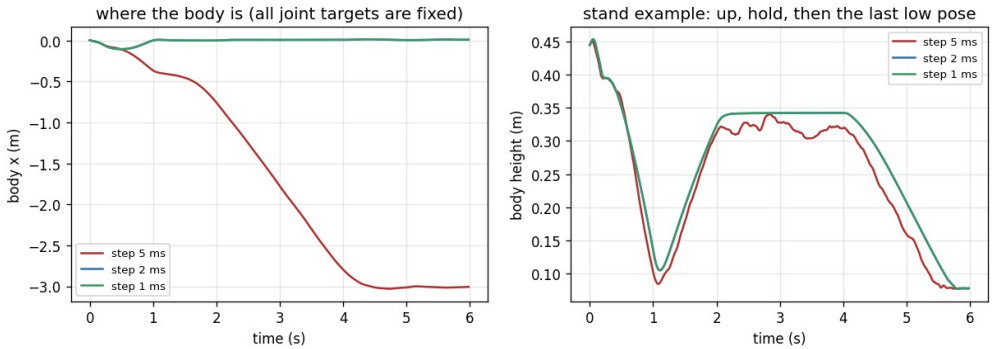


Figure 2. *The same joint targets with three physics time steps. Only the 5 ms step, the simulator's default, makes the body slide.*

2 Question 3: Getting Familiar with the Dog Apps

2.1 Review of the five demos

I read each project to see what it does and what it needs (Table 2). Four of the five need the real robot or an NVIDIA GPU. They also show where the course is going. go2_robot and the SLAM and Nav2 demo are the mapping and navigation parts of the building-delivery robot I proposed in my earlier Unitree assignment.

Table 2. The five demo projects.

Project	What it does	What it needs	Ran here
go2_robot	ROS 2 driver: robot model, odometry, point cloud, joint states, rviz, velocity commands	real Go2, ROS 2 Humble	no
go2-webrtc	web page that connects to the Go2 over WebRTC, sends commands and shows its video	real Go2 on the same network and its login token	no
unitree-go2-slam-nav2	SLAM mapping and Nav2 autonomous navigation demos	real Go2 with LiDAR and a depth camera, ROS 2	no
Go2-Dynamic-Inspection	"Go2 Planner Suite": CUDA-accelerated navigation with a Gazebo simulation	ROS 2 Humble and an NVIDIA GPU	no
Go2Py	Python interface for the Go2, MuJoCo simulator and trained RL walking controllers	nothing extra for simulation	yes

2.2 Running Go2Py

Two things stopped Go2Py on my Mac. I fixed both without editing its files. Its walk-these-ways checkpoint was saved with GPU tensors, which cannot load without CUDA, so a small module maps them to the CPU before Go2Py is imported. Its "front_camera" points out of the dog's left side, not forward. The demo does not need the camera, and I turned it forward for Question 4.

The trained controller followed every command closely (Figure 3). Commanded at 0.6 m/s forward it walked at 0.60 m/s, and it also matched 0.8 rad/s turning, 0.3 m/s sideways and 0.3 m/s backward within 0.01. Its body never dropped below 0.23 m. The same run can be watched live with mjpython demo_walk.py --view.

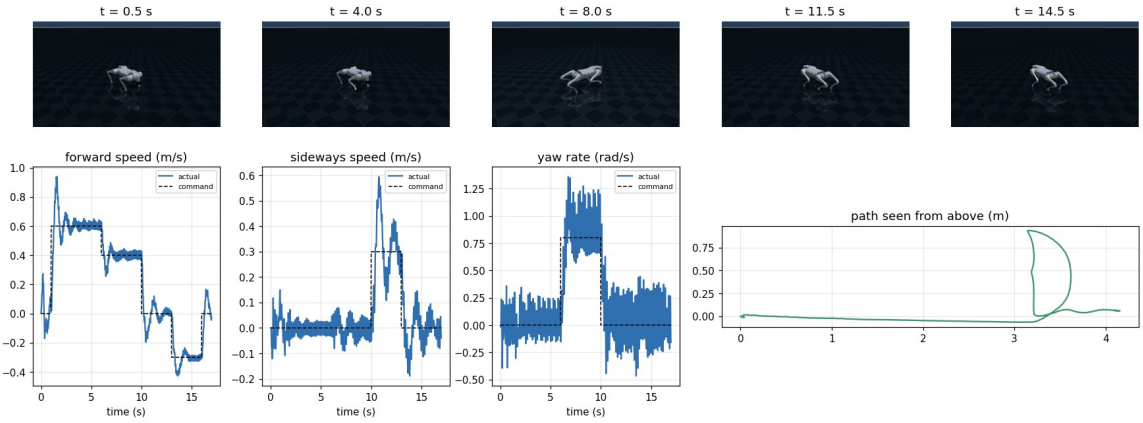


Figure 3. *Go2Py's walk-these-ways controller in MuJoCo. Top: the tracking camera. Bottom: commanded and actual speeds, and the path seen from above.*

3 Question 4: Dog Data and Simulation

3.1 How I plan to collect real data

The suggested repository, unitree_go2_create_dataset, records the real robot's joint states and foot forces from the lf/lowstate topic into a ROS 2 bag. It does not record images, and it does not make synthetic data. "synthetic" is only the name of one of its topics. So I wrote my own collector, lab/collect_real.py. It only

listens and never sends a command. It saves front camera frames through VideoClient, the 12 joint angles and speeds, foot forces, IMU and battery from rt/lowstate, and the robot's own velocity estimate. This is the same kind of record my simulator makes, so the same analysis can run on real frames. The lab steps are in lab/LAB_CHECKLIST.md.

3.2 Synthetic data

Go2Py's default scene is an empty dark floor, so I built a test field on top of its Go2 model (Figure 4). It has soil with texture, sunlight, ten weeds and five objects chosen to test the methods. The red box has strong color. The gray box has almost none, to test hue. A box the color of the soil was meant to fool GrabCut. The dog walked a fixed route with Go2Py's controller: forward, a fast turn left and back, a sidestep, and forward again. Every 0.1 s the front camera saved an image, the true label of every pixel and the depth, 170 frames in total.

The objects do not move and the camera pose is known exactly. So the true optical flow of every pixel can be computed from the depth and the two camera poses. My earlier assignments never had this ground truth. As a check, the DIS flow method agrees with it to about 2 pixels on slow motion.

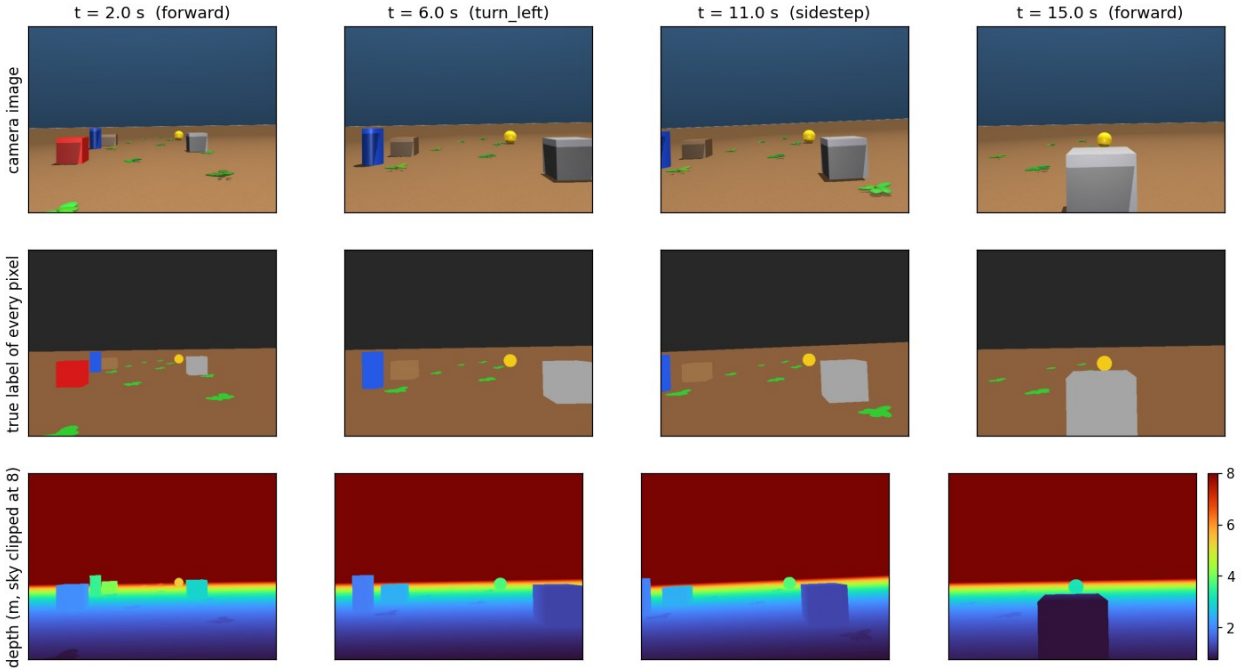


Figure 4. *The synthetic data at four moments of the run. Every image comes with the true object under each pixel and its depth.*

3.3 The five methods on the synthetic data

Color space. I scored each channel alone on how well it separates weed pixels from soil pixels, using the AUC (1.0 is perfect). Lab a, Cr and hue reached 1.000. The green channel G reached only 0.741, because brown soil also has a lot of green. Lightness L was 0.544, almost useless. When I moved every pixel by one RGB unit, the hue of the red box changed by 1.1 degrees and the hue of the gray box by 27.2 degrees. This is the same warning as in the color review, and the same result as my Reading 2.

Gaussian pyramid. I tracked 20,766 corners with pyramidal Lucas-Kanade and compared each result with the true flow (Table 3). Without a pyramid, the share tracked within 1 pixel fell from 62% for motion under 2 pixels to 7% at 20 to 40 pixels. Above 40 pixels, which is the fast turn, it was 0%. With three extra levels it stayed at 60 to 67% up to 20 pixels and still reached 30% above 40 pixels. It is my Homework 1 result, an error of 1.77 pixels for a 5-pixel shift, seen again on camera images.

Table 3. Share of corners tracked to within 1 pixel of the true position, by true motion between frames.

Pyramid levels	0-2 px	2-5 px	5-10 px	10-20 px	20-40 px	40-90 px
none	62%	55%	43%	28%	7%	0%
1 extra	61%	61%	60%	59%	26%	8%
2 extra	61%	60%	61%	66%	45%	18%
3 extra	61%	60%	61%	67%	49%	30%
corners	689	4,303	5,011	3,878	2,514	4,001

Optical flow. DIS had a mean error of 2.3 pixels when walking forward, 12.9 pixels in the fast turn and 2.0 pixels in the sidestep (Table 4, Figure 5). Farneback failed almost completely, with errors close to the size of the motion itself. To find out why, I moved one frame by a known (3, 2) pixels. On the soil Farneback returned (0.01, 0.01), while DIS returned (3.01, 2.00). The OpenCV source explains it. Each Farneback update divides by the determinant of a 2 by 2 matrix plus 0.001 (optflowgf.cpp: idet = 1/(g11 g22 − g12^2 + 1e-3)). The simulated soil changes by only about half a gray level per pixel, so the determinant is tiny and the 0.001 pushes every update toward zero. When I multiplied the soil's contrast by 2, 4 and 8, the estimate of the 3-pixel shift grew from 0.16 to 2.18 to 2.94 pixels. It is the same matrix as A^TA in Homework 1.

Table 4. Mean end-point error of dense optical flow against the true flow, in pixels.

Motion	True motion (median)	Farneback	Farneback + CLAHE	DIS
forward	6.8	9.30	8.33	2.31
fast turn	58.4	53.73	54.23	12.90
sidestep	14.1	12.82	11.94	1.96
time per frame pair		29 ms	30 ms	12 ms

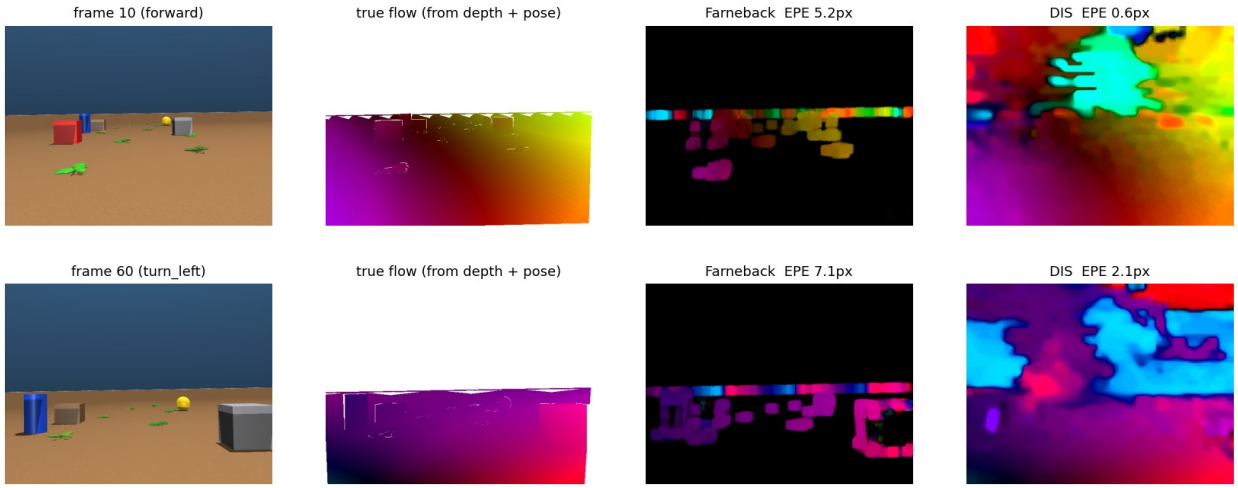


Figure 5. *True flow and two estimates for a forward step (top) and the fast turn (bottom). Hue is direction and brightness is speed. Farneback's output is almost black on the soil, which means almost no motion.*

GrabCut. I gave GrabCut a rectangle 15% larger than each object (Figure 6). The IoU was 0.98 for the blue cylinder, 0.97 for the soil box and 0.96 for the red box. It was 0.92 for the ball, 0.88 for the gray box and 0.67 for the weed. The soil box did not fool it. Its sides were in shade, which gave a contrast-to-noise ratio of 3.6, above the limit of about 2.2 I measured in Reading 2. All six objects were above that limit. GrabCut never missed a pixel of any object. Every error was extra pixels, and they were shadows. The extra soil under the weed had a brightness of 86, while the rest of the soil in its rectangle had 136. The shadow is inside the rectangle but not outside it, so the background model never learned it. That is the third failure case in Section 5 of the GrabCut paper.

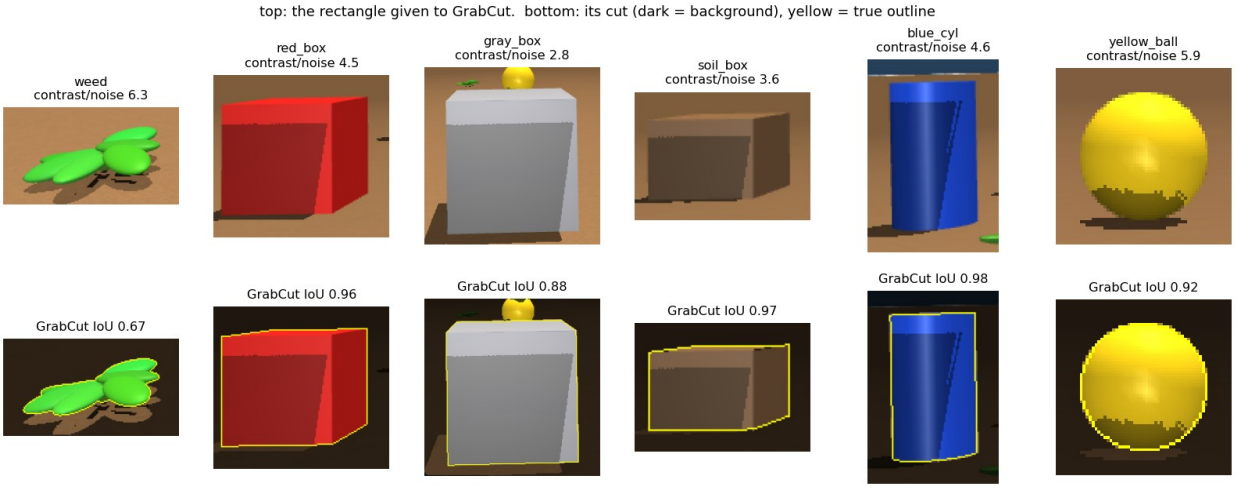


Figure 6. *GrabCut on each object. Top: the image inside the rectangle. Bottom: GrabCut's result, with the background darkened and the true outline in yellow. The extra pixels are the shadows.*

Supervised learning. I trained six classifiers from Kotsiantis's review to label weed pixels (Table 5). They used the 25 pixel features from Reading 3 and the simulator's exact labels. Training used 88 frames and testing used 60 later frames seen from new viewpoints. All six reached an F1 between 0.989 and 0.999. Naive Bayes was lowest. k-nearest neighbors needed about 3 s per image, against 9 ms for the decision tree. Both match the accuracy and speed stars in the review's Table 4. At its strongest penalty the lasso kept only three channels, ExG, R and Lab a, and still reached F1 0.986. Lab a was kept at every penalty, as in Reading 3. On real photos with weak labels, my best pixel F1 in Reading 3 was 0.786. The gap shows how much easier clean synthetic images with exact labels are.

Table 5. Weed-pixel classifiers on 36,000 test pixels from later frames.

Classifier	Family (Kotsiantis)	F1	Time per 640x480 image
Lasso logistic regression	statistical / linear	0.998	13 ms
Decision tree	logic-based	0.999	9 ms
Naive Bayes	statistical	0.989	46 ms
k-nearest neighbors	instance-based	0.999	3,064 ms
Linear SVM	support vector machine	0.999	20 ms
Neural network (MLP)	perceptron-based	0.999	56 ms

3.4 Dog simulation with unitree_rl_gym

Training in unitree_rl_gym needs Isaac Gym, which runs only on Linux with an NVIDIA GPU. The official repository also has no Go2 policy, only humanoids. A pull request (#62, fork UESTC-REVERIE/unitree_rl_gym_go2) adds a Go2 policy trained in Isaac Gym and deploy_go2.py, which runs it in MuJoCo. Running a trained policy does not need Isaac Gym, so it works on my Mac.

Reading deploy_go2.py, I noticed that it gives the policy the body's linear velocity in world coordinates, while Isaac Gym trains with the velocity in the robot's own body frame. The two are the same only while the robot faces its starting direction. I ran the same commands twice, once with the original code and once with the velocity turned into the body frame (Table 6, Figure 7). Both stood and walked forward with an error of about 0.04 m/s. When the command asked the dog to turn while walking, the original version's turning error rose to 1.0 rad/s. The dog fell to 0.133 m and never got up. With the body-frame velocity the errors stayed between 0.05 and 0.07, and the body never went below 0.27 m.

Table 6. RMS tracking error of the unitree_rl_gym Go2 policy, 1 s after each new command.

Phase	Velocity given in	Forward (m/s)	Sideways (m/s)	Turn (rad/s)	Lowest body (m)
walk forward 0.5 m/s	world frame (original)	0.038	0.054	0.047	0.308
walk forward 0.5 m/s	body frame (fixed)	0.041	0.042	0.043	0.311
walk 0.5 m/s + turn 0.6 rad/s	world frame (original)	0.543	0.502	1.003	0.133 (fell)
walk 0.5 m/s + turn 0.6 rad/s	body frame (fixed)	0.048	0.057	0.062	0.309

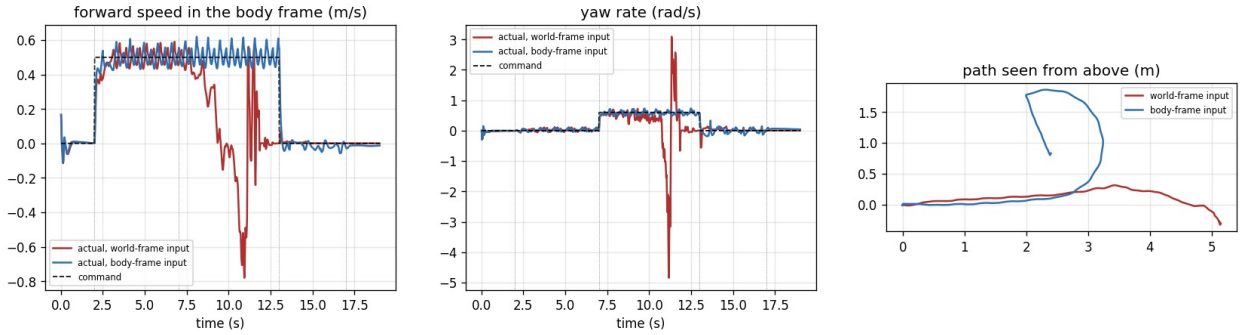


Figure 7. *The same commands with the velocity given in the world frame (red, as in deploy_go2.py) and in the body frame (blue). The red run falls soon after the turn starts.*

3.5 What I learned

Most of my time went into problems that no paper mentions. A camera pointing the wrong way, a time step that is too coarse and a velocity in the wrong frame all looked like the method failing. In each case one measurement against a known answer found the real cause. Simulation made that possible, because it gives exact labels, depth and poses. It also showed its limits. The synthetic soil was too smooth for Farneback, and the weeds were too easy for every classifier. The next step is to collect real frames with lab/collect_real.py and run the same analysis on them.

References

The papers behind the five methods are listed in the review paper (Question 1). The software used here:

[1] Unitree Robotics, “unitree_sdk2_python,” GitHub. https://github.com/unitreerobotics/unitree_sdk2_python

[2] Unitree Robotics, “unitree_mujoco,” GitHub. https://github.com/unitreerobotics/unitree_mujoco

[3] Eclipse Foundation, “Cyclone DDS,” releases/0.10.x. https://github.com/eclipse-cyclonedds/cyclonedds

[4] R. Khorrambakht and B. Dai, “Go2Py,” GitHub. https://github.com/machines-in-motion/Go2Py

[5] Unitree Robotics, “unitree_rl_gym,” GitHub, and pull request #62 (fork UESTC-REVERIE/unitree_rl_gym_go2). https://github.com/unitreerobotics/unitree_rl_gym

[6] S. Zhao, “unitree_go2_create_dataset,” GitHub. https://github.com/ShijieZhao36/unitree_go2_create_dataset

[7] OpenCV, “optflowgf.cpp,” Farneback optical flow source, 4.x branch. https://github.com/opencv/opencv/blob/4.x/modules/video/src/optflowgf.cpp

[8] Demo projects reviewed in Table 2: github.com/Unitree-Go2-Robot/go2_robot, github.com/tfoldi/go2-webrtc, github.com/h-naderi/unitree-go2-slam-nav2, github.com/YasiruDEX/Go2-Dynamic-Inspection.